```
after_update_commit -> do
  Turbo::StreamsChannel.broadcast_stream_to(
    concert,
    content: {seat: id, status: status}.to_json
  )
end
```

They then both render default Turbo Stream views. Both views are the same; I'll only show one:

appendix_hotwire/01/app/views/seats/update.turbo_stream.erb

```
<%= turbo_stream.replace(
      dom_id(@row),
      partial: "rows/row",
      locals: {row: @row, concert: @concert, user: @user}
    ) %>
<%= turbo_stream.replace(
      "subtotal",
      partial: "venues/subtotal",
      locals: {concert: @concert, user: @user}
    ) %>
```

This triggers two Turbo Stream replace actions. The second is the same update of the subtotal that we did before, though this time we would expect the total to change. The first we take advantage of having used ActiveModel, and we just replace the row that the seats are in—given our application logic, no other part of the venue grid outside that row will change.

Finally, the Clear Tickets button requires a little bit of Stimulus because it needs to pass the tickets-to-buy count to the server so the server can hand down a cleared venue with the right invalid tickets. I've got it set up as a form submit that reaches to the existing pull-down menu to get its value before submitting:

appendix_hotwire/01/app/javascript/controllers/clear_all_controller.js

```
import { Controller } from "@hotwired/stimulus"
import "form-request-submit-polyfill"

export default class ClearAllController extends Controller {
  static targets = ["form", "hiddenField"]
```